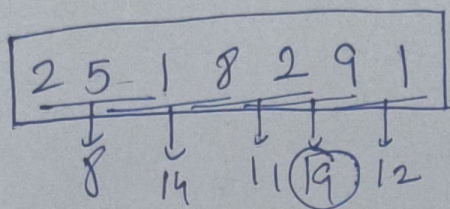


Sliding Window

Maximum Sum Subarray of size k



$$k = 3$$

Window size $\rightarrow$ start = i , end = j

$$\text{Size} = j - i + 1$$

```
Code:
int ans = 0;
int i = 0;
int j = 0;
int sum = 0;
While (j < size) {
    sum = sum + arr[j];
    if (j - i + 1 < k) {
        j++;
    }
    else if (j - i + 1 == k) {
        ans = max(sum, ans);
        sum = sum - arr[i];
        i++;
        j++;
    }
}
```

}

First Negative Number in every Window of size k

arr[]:

12	-1	-7	8	15	30	16	28
----	----	----	---	----	----	----	----

$k = 3$

-1, -1, -7, -15, -15, 0

list <int> l;

while (j < size) {

if (arr[i] < 0) l.push-back(arr[i]);

if (j - i + 1 < k) j++;

else if (j - i + 1 == k) {

if (l.size() == 0) cout << "0";

else {

or v.push-back(0);

v.push-back(l.front());

}

if (arr[i] == l.front()) {

l.pop-front();

}

}

eg: 12 -1 -7 8 -15 30 16 28

U keep adding in list if arr[i] is negative

{-1, -7}

For returning when win size == k,

print l.front() or

0 if list empty

Then remove front element if equal to arr[i].
slide window.

Count no. of anagrams

$$S = \boxed{b o a} \times \times \boxed{o n b} \times a \boxed{o b n}$$

ptr = ~~boa~~ $\rightarrow$ same freq. as that of pattern
 $K = \text{ptr.size}()$;

$b \rightarrow 1$
 $o \rightarrow 1$
 $n \rightarrow 1$ } keep decreasing as you encounter them.

$$\text{Count} = \text{mp.size}() \\ = 3$$

$\rightarrow$ no. of elements whose freq. in map $\neq 0$.

boa

$b \rightarrow 0$
 $o \rightarrow 0$
 $n \rightarrow 1$

$$\text{Count} = 3 \neq 1$$

$\text{Count} \neq 0 \rightarrow$ no ans

oax

$b \rightarrow 0$
 $o \rightarrow 0$
 $n \rightarrow 1$

$\text{Count} \neq 0 \rightarrow$ no ans

onb

$b \rightarrow 0$
 $n \rightarrow 0$
 $o \rightarrow 0$

$\text{Count} = 0 \rightarrow$ ans $\checkmark$

Maximum of all subarrays of size K

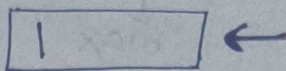
$K=3$

1	3	-1	-3	5	3	6	7
	3			5		6	7
		3			5		7
			5			6	
				5			7
					6		
						6	7
							7

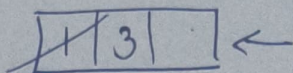
Using a doubly ended queue.

0 1 2 3 4 5 6 7 8
1 3 -1 -3 5 5 3 6 7

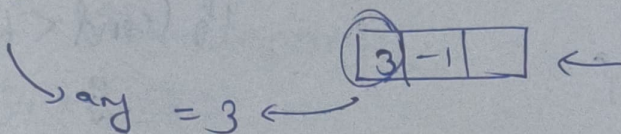
$j = 0$



$j = 1$



$j = 2$

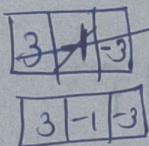


When inserting a larger element, remove all those smaller than it.

you can add a smaller number

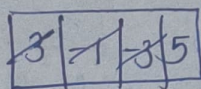
$j = 3$

$\rightarrow \text{any} = 3$



keep popping from right till the number being inserted is bigger than inside.

$j = 4$



Removal $\rightarrow$ if $\text{list.front} == \text{arr}[i] \rightarrow \text{pop_front}()$

Code: Calculation part

while ($\text{l.size}() > 0$ && $\text{l.back}() < \text{arr}[j]$) {
 $\text{l.pop_back}()$;

j
 $\text{l.push_back}(\text{arr}[j])$

any $\leftarrow$ calculation part

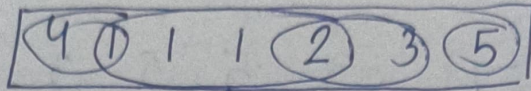
$\text{any.push_back}(\text{l.front}());$

slide window part

if ($\text{l.front}() == \text{arr}[j]$) {
 $\text{l.pop_front}()$;

Longest Subarray of Sum K

K=5



Fixed Window size

```
While (j < size) {
```

```
    Calculations
```

```
    if (win size < K) {  
        j++;
```

```
    } else if (win size == K) {
```

```
        ans ← calculation
```

```
        Calculation remove i
```

```
        Window size maintain and slide
```

```
    }
```

Variable Window size

```
While (j < size) {
```

```
    Calculations
```

```
    if (cond < K) {  
        j++;
```

```
    } else if (cond == K) {
```

```
        ans ← calculation
```

```
        j++;
```

```
    } else if (cond > K) {
```

```
        While (cond > K) {
```

```
            remove calculation
```

```
            for i
```

```
            i++;
```

Longest Substring With K Unique Characters

s: a a b a c b e b e b e

K=3

3 unique character

Create a mp with ~~ele~~ char → freq.

keep checking mp.size() == K.

When freq becomes zero for a character,
remove it from map.

mp.erase(s[i])

Longest Substring without Repeating Characters

$s =$ p w w k e w any $\rightarrow 3$

Create map. character $\rightarrow$ freq : Compare it with Window size.

Minimum Window Substring

$S =$ "timetopractice"

$t =$ "t oc"

String s Substring should contain atleast all characters of t in their ~~required~~ given frequency, order doesn't matter.

o a b c t ✓

t c c c o o a ✓

t a a a c ✗

Start with mp

$t \rightarrow 1$

$o \rightarrow 1$

$c \rightarrow 1$

as they appear, reduce the freq. if not, increase it.

Count = 3

$\hookrightarrow$ no. of non zero elements

Sum of array When zeros are there

Subarrays with $\text{sum} == \text{goal}$

= subarrays with $\text{sum} \leq \text{goal}$

- subarrays with $\text{sum} \leq \text{goal} - 1$

Code:

```
int atmost (vector<int> & nums, int k) {
```

```
if (k < 0) return 0;
```

```
int i = 0; sum = 0; count = 0;
```

```
for (int i = 0; i < nums.size(); i++) {
```

```
    sum += nums[i];
```

```
    while (sum > k) {
```

```
        sum -= nums[i];
```

```
        i++;
```

```
    }
```

```
    count += (i - i + 1);
```

```
}
```

```
return count;
```

```
}
```

For negatives in array

```
int subarraySum(vector<int> &nums, int k) {
```

```
    unordered_map<int, int> mp;
```

```
    mp[0] = 1;
```

```
    int sum = 0; count = 0;
```

```
    for (int num : nums) {
```

```
        sum += num;
```

```
        if (mp.find(sum - k) != mp.end()) {
```

```
            count += mp[sum - k];
```

```
        } mp[sum]++;
```

```
    } return count;
```

```
}
```

```
int numSubarraysWithSum
```

```
(vector<int> &nums, int goal)
```

```
{
```

```
    return atMost(nums, goal)
```

```
    - atMost(nums, goal - 1);
```

```
}
```

```
}
```

nums = [1, -1, 5, -2, 3]

k = 3

prefix sum by index $\rightarrow$

-1 $\rightarrow$ 0 $\rightarrow$ Base

0 $\rightarrow$ 1 cond.

1 $\rightarrow$ 0

2 $\rightarrow$ 5

3 $\rightarrow$ 3

4 $\rightarrow$ 6

Sum of = Prefix Sum [j]

subarray

- Prefix Sum [i]

~~i~~
i+1 $\rightarrow$ j

$$1 \rightarrow 4 = PS[4] - PS[0]$$

$$= 6 - 1 = 5$$

When no. of
sub arrays required.

$$0 \rightarrow 4 = PS[4] - PS[-1] \rightarrow ??$$

$$= 6 - 0 = 6$$

Store ^{prefix} sum $\rightarrow$ freq. (0 $\rightarrow$ 1 base case
base -1 index)

0 $\rightarrow$ 2

1 $\rightarrow$ 1

3 $\rightarrow$ 1

5 $\rightarrow$ 1

6 $\rightarrow$ 1

When max length is required

store prefix sum $\rightarrow$ ~~last~~ first index

0 $\rightarrow$ -1

1 $\rightarrow$ 0

3 $\rightarrow$ 3

5 $\rightarrow$ 2

6 $\rightarrow$ 4